A3 – Entrega 1: Scanner de Quadro com IA

04/11/2026

Este relatório propõe um sistema que transforma a foto torta de um quadro de aula em uma imagem reta e legível e, em seguida, usa IA para resumir e explicar o conteúdo, unindo homografia e convolução (Álgebra Linear) e flood fill (Recorrência e Indução).

Campo	**Informação**
Instituição	UNIFACS
Disciplina	Estruturas Matemáticas e Computação Gráfica
Professores	Alberlan Lopes, Inácio Borges, Norberto Maciel
Semestre	2026.2
Entrega	1 — Fase de Modelagem e Definição Teórica
Integrantes	Artur Jansen Almeida de Souza; Davi Jansen Almeida de Souza; Luiz

1. Situação-problema

Em sala de aula, é comum o aluno fotografar o quadro em vez de copiar o conteúdo. Essas fotos quase sempre saem ruins: tiradas de lado (o quadro aparece como um trapézio), com reflexo das lâmpadas, sombra, manchas de apagador e letras pequenas. Depois, no momento de estudar, o aluno tem dezenas de fotos difíceis de ler e sem organização.

O grupo propõe um **scanner de quadro com IA**: o aluno envia a foto e o sistema:

1. Localiza o quadro na imagem e corrige a perspectiva, deixando-o reto e retangular.
2. Corrige a iluminação (sombras e reflexos) e realça a escrita.
3. Remove manchas e resíduos de apagador, mantendo apenas a escrita.
4. Separa a escrita em blocos de conteúdo.
5. Envia a imagem limpa para um modelo de IA, que extrai o texto, gera um resumo e explica o tema da aula.

2. Justificativa de relevância

O projeto resolve um problema que os próprios alunos vivem todo dia, usando só o celular.

- **Legibilidade:** fotos tortas e com reflexo viram imagens retas e limpas, como se o quadro tivesse sido escaneado.

- **Estudo mais eficiente:** em vez de reler fotos confusas, o aluno recebe o conteúdo organizado, resumido e explicado.
- **Acessibilidade:** ajuda quem senta longe do quadro, tem baixa visão ou perdeu parte da aula.
- **Robustez a fotos reais:** a correção de perspectiva e de iluminação funciona com fotos tiradas de qualquer ângulo e em salas com luz irregular.
- **Aderência à disciplina:** toda a etapa visual depende diretamente do conteúdo do curso — convolução e homografia (matrizes) e flood fill (recorrência com prova por indução). A IA atua apenas no final, sobre a imagem já tratada.

3. Objetivos

Objetivo geral: desenvolver um protótipo que receba a foto de um quadro de aula, gere uma imagem retificada e limpa e produza um resumo e uma explicação do conteúdo, com cada etapa visual fundamentada matematicamente.

Objetivos específicos:

1. Modelar a imagem digital como matriz e aplicar filtros de convolução para suavização, detecção de bordas e correção de iluminação.
2. Calcular a homografia entre o quadro fotografado e um retângulo de destino, resolvendo um sistema linear em coordenadas homogêneas.
3. Binarizar a escrita com limiarização adaptativa.
4. Segmentar a escrita com flood fill, formulado por recorrência e com correção provada por indução, removendo manchas e agrupando blocos de conteúdo.
5. Integrar um modelo de IA que extraia o texto, resuma e explique o tema da aula.

4. Técnicas de processamento visual

O pipeline tem oito etapas em sequência; as sete primeiras são de visão computacional e estão formalizadas nas seções 5 e 6.

Etapa	Técnica	Fundamento matemático	Seção
1. Conversão para tons de cinza	Combinação linear dos canais RGB	Produto escalar	5.2
2. Suavização	Filtro Gaussiano 3×3	Convolução discreta	5.3
3. Detecção de bordas	Operador de Sobel	Convolução e gradiente	5.3
4. Localização dos cantos do quadro	Contorno de maior área com 4 vértices	Geometria no plano	5.4
5. Correção de perspectiva	Homografia (warp)	Matriz 3×3 em coordenadas homogêneas	5.4
6. Correção de iluminação e binarização	Estimativa de fundo + limiar adaptativo	Convolução com máscara grande	5.5 e 5.6
7. Limpeza e segmentação da escrita	Flood fill + filtro por área	Recorrência e indução	6
8. Resumo e explicação	Modelo de IA multimodal	— (módulo complementar)	7

5. Modelagem matemática — Álgebra Linear

Toda a etapa visual trata a imagem como matriz e cada operação como produto escalar, convolução ou produto matriz-vetor.

5.1 Imagem como matriz

Uma imagem colorida de M linhas e N colunas é um tensor de três matrizes (canais R, G e B). Cada pixel (x, y) é um vetor de intensidades entre 0 e 255:

$$I \in \{0, \ldots, 255\}^{M \times N \times 3}, \qquad \mathbf{p}(x, y) = \begin{bmatrix} R \\ G \\ B \end{bmatrix}$$

5.2 Conversão para tons de cinza

A intensidade em cinza é o produto escalar entre o vetor do pixel e um vetor de pesos (padrão ITU-R BT.601, usado pelo OpenCV):

$$f(x,y) = \mathbf{w}^T \mathbf{p}(x,y) = 0{,}299\,R + 0{,}587\,G + 0{,}114\,B$$

5.3 Filtros espaciais por convolução

Um filtro é uma máscara w de tamanho (2a+1) × (2b+1). A imagem filtrada g é obtida pela convolução discreta:

$$g(x,y) = (w * f)(x,y) = \sum_{s=-a}^{a} \sum_{t=-b}^{b} w(s,t)\, f(x-s,\, y-t)$$

Suavização (Gaussiano 3×3): reduz o ruído da foto antes da detecção de bordas. A soma dos pesos é 1, então o brilho médio é preservado.

$$w_G = \frac{1}{16}\begin{bmatrix} 1 & 2 & 1 \\ 2 & 4 & 2 \\ 1 & 2 & 1 \end{bmatrix}$$

Detecção de bordas (Sobel): aproxima as derivadas parciais da imagem nas direções x e y. É usada para achar a moldura do quadro contra a parede.

$$G_x = \begin{bmatrix} -1 & 0 & 1 \\ -2 & 0 & 2 \\ -1 & 0 & 1 \end{bmatrix}, \qquad G_y = G_x^T = \begin{bmatrix} -1 & -2 & -1 \\ 0 & 0 & 0 \\ 1 & 2 & 1 \end{bmatrix}$$

$$\|\nabla f\| \approx \sqrt{(G_x * f)^2 + (G_y * f)^2}$$

Propriedade de Álgebra Linear explorada: a máscara de Sobel é separável, ou seja, é o produto externo de dois vetores. Isso reduz o custo por pixel de 9 para 6 multiplicações.

$$G_x = \begin{bmatrix} 1 \\ 2 \\ 1 \end{bmatrix} \begin{bmatrix} -1 & 0 & 1 \end{bmatrix}$$

5.4 Correção de perspectiva por homografia

Uma foto tirada de lado distorce o retângulo do quadro em um quadrilátero qualquer. A homografia é a transformação projetiva que leva esse quadrilátero de volta a um retângulo. Em coordenadas homogêneas, ela é uma matriz H de dimensão 3 × 3:

$$\begin{bmatrix} x' w \\ y' w \\ w \end{bmatrix} = \underbrace{\begin{bmatrix} h_{11} & h_{12} & h_{13} \\ h_{21} & h_{22} & h_{23} \\ h_{31} & h_{32} & 1 \end{bmatrix}}_{H} \begin{bmatrix} x \\ y \\ 1 \end{bmatrix}$$

Voltando para coordenadas cartesianas (divisão por w):

$$x' = \frac{h_{11}x + h_{12}y + h_{13}}{h_{31}x + h_{32}y + 1}, \qquad y' = \frac{h_{21}x + h_{22}y + h_{23}}{h_{31}x + h_{32}y + 1}$$

Por que coordenadas homogêneas: translação, escala e rotação viram todas produtos de matrizes 3 × 3, e podem ser compostas por multiplicação. São casos particulares de H:

$$T = \begin{bmatrix} 1 & 0 & t_x \\ 0 & 1 & t_y \\ 0 & 0 & 1 \end{bmatrix}, \quad S = \begin{bmatrix} s_x & 0 & 0 \\ 0 & s_y & 0 \\ 0 & 0 & 1 \end{bmatrix}, \quad R_\theta = \begin{bmatrix} \cos\theta & -\sin\theta & 0 \\ \sin\theta & \cos\theta & 0 \\ 0 & 0 & 1 \end{bmatrix}$$

Cálculo de H: H tem 8 incógnitas. Multiplicando as equações acima pelo denominador, cada par de pontos (x, y) → (x', y') gera duas equações lineares:

$$h_{11}x + h_{12}y + h_{13} - h_{31}xx' - h_{32}yx' = x'$$
$$h_{21}x + h_{22}y + h_{23} - h_{31}xy' - h_{32}yy' = y'$$

Com os 4 cantos do quadro (detectados na etapa 4) e os 4 cantos do retângulo de destino, (0, 0), (W−1, 0), (W−1, H−1) e (0, H−1), obtém-se um sistema linear 8 × 8, A h = b:

$$\begin{bmatrix} x_1 & y_1 & 1 & 0 & 0 & 0 & -x_1x'_1 & -y_1x'_1 \\ 0 & 0 & 0 & x_1 & y_1 & 1 & -x_1y'_1 & -y_1y'_1 \\ \vdots & & & & & & & \vdots \\ 0 & 0 & 0 & x_4 & y_4 & 1 & -x_4y'_4 & -y_4y'_4 \end{bmatrix} \begin{bmatrix} h_{11} \\ h_{12} \\ h_{13} \\ h_{21} \\ h_{22} \\ h_{23} \\ h_{31} \\ h_{32} \end{bmatrix} = \begin{bmatrix} x'_1 \\ y'_1 \\ \vdots \\ y'_4 \end{bmatrix}$$

O sistema tem solução única quando nenhum trio dos 4 pontos é colinear (det A ≠ 0), o que vale para os cantos de um quadro. A razão W/H do destino é estimada pelas distâncias entre os cantos detectados.

Mapeamento inverso: para preencher a imagem retificada sem buracos, cada pixel de destino (u, v) busca sua origem por H^{-1}, e a intensidade é obtida por interpolação bilinear dos 4 vizinhos.

$$\begin{bmatrix} x\,w \\ y\,w \\ w \end{bmatrix} = H^{-1} \begin{bmatrix} u \\ v \\ 1 \end{bmatrix}$$

5.5 Correção de iluminação

O reflexo das lâmpadas e as sombras variam lentamente pelo quadro, enquanto a escrita muda bruscamente. Por isso o fundo b é estimado por uma convolução com uma máscara de média grande (k × k, com k bem maior que a espessura do traço), que “apaga” a escrita e mantém só a iluminação:

$$b = w_k * f, \qquad w_k = \frac{1}{k^2}\,\mathbf{1}_{k\times k}$$

A imagem corrigida é a razão, pixel a pixel, entre a imagem e o fundo estimado (normalizada para 0 a 255):

$$\hat{f}(x,y) = 255 \cdot \min\!\left(1,\ \frac{f(x,y)}{b(x,y)}\right)$$

Em regiões com reflexo, f e b são altos e a razão volta perto de 1 (fundo branco); onde há escrita, f fica bem abaixo de b e o traço é realçado. Para quadro negro, aplica-se o mesmo sobre a imagem invertida 255 − f.

5.6 Binarização adaptativa

Um único limiar para o quadro inteiro falha com luz irregular. O limiar adaptativo usa a média local, obtida também por convolução, menos uma constante C:

$$B(x,y) = \begin{cases} 1, & \hat{f}(x,y) < (w_m * \hat{f})(x,y) - C \\ 0, & \text{caso contrário} \end{cases}$$

Em que w_m é uma máscara de média m × m. O resultado é a matriz binária B, em que 1 indica escrita e 0 indica fundo.

6. Modelagem matemática — Recorrência e Indução

A limpeza e a organização da escrita são feitas por flood fill, definido por uma relação de recorrência; provamos por indução que ele termina, marca exatamente uma região conexa e custa 4n + 1 chamadas.

6.1 Definições

- Domínio D: o conjunto de pixels da imagem retificada, de dimensão W × H.
- Vizinhança-4 de um pixel p = (x, y):

$$N_4(p) = \{(x+1, y),\ (x-1, y),\ (x, y+1),\ (x, y-1)\}$$

- Componente conexa C(p0): conjunto dos pixels q ∈ D com B(q) = 1 ligados a p0 por um caminho de vizinhos-4 que só passa por pixels com B = 1. Cada letra, traço ou mancha é uma componente.
- Distância d(q): comprimento do menor desses caminhos de p0 até q.

6.2 Formulação por recorrência

Seja V o conjunto de pixels já visitados. O flood fill é a função:

$$FF(p, V) = \begin{cases} V, & p \notin D \ \vee \ B(p) = \\ FF\big(p_4, FF(p_3, FF(p_2, FF(p_1, V \cup \{p\})))\big), & \text{caso contrário} \end{cases}$$

Em que p1, p2, p3 e p4 são os quatro vizinhos de N4(p).

6.3 Demonstrações

Teorema 1 (terminação). FF(p0, ∅) termina para qualquer p0.

Prova. Toda chamada que não cai no caso base insere em V um pixel que ainda não estava lá. Como V ⊆ D e D é finito, existem no máximo |D| chamadas desse tipo; cada uma gera exatamente 4 chamadas filhas. Logo o número total de chamadas é finito. ∎

Teorema 2 (correção). Se B(p0) = 1, então FF(p0, ∅) = C(p0).

Parte (i) — tudo que é marcado pertence à região. Indução na ordem k em que os pixels entram em V.

- Caso base (k = 1): o primeiro pixel inserido é p0, e p0 ∈ C(p0).
- Passo indutivo: suponha que os k primeiros pixels inseridos estão em C(p0). O pixel k+1, chamado q, foi inserido numa chamada FF(q, ·) feita por um pixel q’ já em V, com q ∈ N4(q’). Pela hipótese, q’ ∈ C(p0). Como q não caiu no caso base, q ∈ D e B(q) = 1. Logo q é vizinho de um pixel da componente e também é escrita: q ∈ C(p0). ∎

Parte (ii) — tudo que pertence à região é marcado. Indução na distância d. Seja P(d): todo q ∈ C(p0) com d(q) = d está em V ao final.

- Caso base (d = 0): o único pixel com distância 0 é p0, inserido na primeira chamada.
- Passo indutivo: suponha P(d). Tome q ∈ C(p0) com d(q) = d + 1. O penúltimo pixel q’ do menor caminho até q tem d(q’) = d, logo q’ ∈ V pela hipótese. Quando q’ foi inserido, sua chamada executou FF(q, ·), pois q ∈ N4(q’). Nessa chamada, ou q já estava em V, ou q ∈ D, B(q) = 1 e q ∉ V, e então q é inserido. Em ambos os casos q ∈ V, e vale P(d + 1). ∎

De (i) e (ii), FF(p0, ∅) = C(p0).

Teorema 3 (custo). Se n pixels são marcados, o número total de chamadas é T(n) = 4n + 1.

Cada pixel marcado acrescenta exatamente 4 chamadas, e a chamada inicial conta 1. Isso dá a recorrência:

$$T(0) = 1, \qquad T(n) = T(n-1) + 4 \quad (n \geq 1)$$

Prova da forma fechada por indução. Caso base: T(0) = 1 = 4·0 + 1. Passo: se T(n) = 4n + 1, então T(n+1) = T(n) + 4 = 4n + 5 = 4(n+1) + 1. ∎

Portanto o algoritmo é O(n), linear no tamanho da região.

Nota de implementação: o Python limita a profundidade de recursão (1.000 chamadas por padrão), e uma região pode ter mais pixels que isso. O protótipo usará a versão iterativa com pilha explícita, que executa a mesma recorrência; as provas acima continuam válidas.

6.4 Rotulação de todas as componentes

Percorrendo os pixels p1, p2, …, p|D| em ordem (linha por linha), o número de componentes encontradas e o conjunto de pixels já rotulados são definidos por recorrência:

$$L_0 = 0,\ V_0 = \varnothing; \qquad \text{se } B(p_k) = 1 \text{ e } p_k \notin V_{k-1}:\ L_k = L_{k-1} + 1,\ V_k = V_{k-1} \cup$$

Prova por indução de que L_k é o número de componentes que têm algum pixel entre p1 e pk. Caso base: L_0 = 0 e nenhum pixel foi visto. Passo: se vale para k, o pixel p(k+1) só aumenta a contagem quando é escrita e não foi rotulado, ou seja, quando é o primeiro pixel visto de uma componente nova; pelo Teorema 2, o flood fill rotula essa componente inteira, então ela não será contada de novo. Logo vale para k+1. ∎

6.5 Remoção de manchas e agrupamento em blocos

Limpeza: resíduos de apagador e pontos de ruído formam componentes muito pequenas. Toda componente com área abaixo de um mínimo a_min é apagada:

$$B'(q) = \begin{cases} 0, & q \in C_i \text{ e } |C_i| < a_{\min} \\ B(q), & \text{caso contrário} \end{cases}$$

O valor de a_min será calibrado com fotos reais, para não apagar pingos de “i” e acentos.

Blocos de conteúdo: para agrupar letras em palavras e linhas em blocos, a imagem limpa é dilatada (convolução com uma máscara de 1s seguida de limiar), o que une traços próximos. Aplicando a rotulação da seção 6.4 sobre a imagem dilatada, cada componente resultante é um

bloco de conteúdo (um tópico, uma fórmula, um esquema), recortado pelo seu retângulo envolvente e enviado em ordem para o módulo de IA.

7. Módulo de IA, ferramentas e próximos passos

A IA entra só no fim do pipeline, como módulo complementar: ela recebe a imagem já retificada, limpa e dividida em blocos, o que melhora muito a leitura do conteúdo. Toda a modelagem matemática avaliada está nas etapas visuais (seções 5 e 6).

Funcionamento do módulo de IA:

1. Cada bloco recortado (seção 6.5) é enviado, em ordem, a um modelo de IA multimodal, que transcreve o texto, as fórmulas e descreve os esquemas.
2. Com a transcrição completa, o modelo gera três saídas: o **tema da aula**, um **resumo** em tópicos e uma **explicação** didática dos conceitos, com exemplos.
3. O aluno recebe a imagem tratada (em PDF) junto com o resumo e a explicação.

Ferramenta	**Uso no projeto**
Python 3	Linguagem do protótipo
OpenCV	Tons de cinza, Gaussiano, Sobel/Canny, contornos, homografia (getPerspectiveTransform, warpPerspective)
NumPy	Operações matriciais, solução do sistema A h = b, correção de iluminação, flood fill próprio
API de modelo de IA multimodal	Transcrição, resumo e explicação do conteúdo
Streamlit	Interface para enviar a foto e ver o resultado

O flood fill e a rotulação serão implementados pelo grupo (sem usar as funções prontas do OpenCV), para que o código siga diretamente as recorrências das seções 6.2 e 6.4.

Próximos passos:

- Entrega 2 (12/11): diagrama de arquitetura, pipeline detalhado, pseudocódigo e especificação das bibliotecas.
- Entrega 3 (29/11): protótipo executável, repositório organizado e README.
- Entrega 4: código final comentado, relatório unificado, slides e vídeo de 10 a 15 minutos.

Referências

- GONZALEZ, R. C.; WOODS, R. E. *Processamento Digital de Imagens*. 3. ed. São Paulo: Pearson Prentice Hall, 2010.
- FRIGERI, S. R.; JR, C. A. C.; ROMANINI, A. *Computação Gráfica*. Porto Alegre: SAGAH, 2018.
- MENEZES, P. B. *Matemática Discreta para Computação e Informática*. Porto Alegre: Bookman, 2013.
- STEIN, C. S.; DRYSDALE, R. L.; BOGART, K. P. *Matemática Discreta para Ciência da Computação*. São Paulo: Pearson, 2013.
- HARTLEY, R.; ZISSERMAN, A. *Multiple View Geometry in Computer Vision*. 2. ed. Cambridge: Cambridge University Press, 2004.
- ZHANG, Z.; HE, L. Whiteboard scanning and image enhancement. *Digital Signal Processing*, v. 17, n. 2, p. 414–432, 2007.